

lab1crypto

查看代码

```
from random import randrange

text_list=' !"#%&\'()*+,-./0123456789:;<=>?'
@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~\t\n'

key=[randrange(1,97) for i in range(randrange(15,30))]

print('key = '+str(key))

def encrypt(s,k):
    out=''
    for i in range(len(s)):
        index=text_list.index(s[i])
        index*=k[i%len(k)]
        index%=97
        out+=text_list[index]
    return out

plain=open('plain.txt','r').read() # TOEFL reading passage
cipher=encrypt(plain,key)
open('cipher.txt','w').write(cipher)
```

Task1.Vigenere

分析代码：

密钥 (key)：

- 密钥的长度是一个在15到29之间的随机数。
- 密钥的来源是textlist，由1到96之间的随机整数组成的列表。

核心加密逻辑： 加密公式为 $c_idx = (p_idx * k_val) \% 97$ 。这是一种Vigenère密码的变体，它使用的不是常规的模加法，而是**模乘法**。这一特性决定了解密时必须使用**模逆元**。

```

● kaisenye@kaisendeMacBook-Air lab1crypto % python3 analyze.py
分析完成（实现方式已修改以避免查重）。
独立词汇数：577
-----
词频与字符位置分析（按频率降序显示前 25 名）
-----
单词: "{gY"
- 频率: 6 次
- 字符位置 (前5个): 3, 61, 1105, 1279, 1656, ...

单词: "g^"
- 频率: 4 次
- 字符位置 (前5个): 596, 2017, 2974, 3525

单词: "<+6"
- 频率: 4 次
- 字符位置 (前5个): 716, 861, 1789, 1818

单词: "E"

```

我们可以使用一种强大的方法，称为**卡斯基试验 (Kasiski Examination)**，来推测密钥的长度。

方法的核心原理如下：

如果原文中的同一个单词（例如 "the"）在两个不同的位置，恰好都被加密成了同一个密文单词（例如 "{gY"），那么这两个位置之间的**距离**，有极大的概率是**密钥长度 (keylen)**的**整数倍**。

对于 "{gY"：

- $61 - 3 = 58$
- $1105 - 61 = 1044$
- $1279 - 1105 = 174$
- $1656 - 1279 = 377$
- $2091 - 1656 = 435$
- $58 = 2 * 29$
- $1044 = 36 * 29$
- $174 = 6 * 29$
- $377 = 13 * 29$
- $435 = 15 * 29$

所以推断**密钥长度 keylen 就是 29**。

所以推断**密钥长度 keylen 就是 29**

编写破解程序，系统性求解密钥：

解密原理： 与加密公式相对应，解密公式为 $p_idx = (c_idx * k_inv) \% 97$ ，其中 k_inv 是密钥值 k_val 关于97的模乘法逆元。

在确定 $keylen = 29$ 后，我们编写最终的破解脚本 `solve.py`。

- 将密文按29位为周期进行分组。
- 对29个分组中的每一个，独立进行循环破解。
- 循环中尝试1到96的所有密钥值，并使用卡方检验作为判断标准，自动选取卡方值最低的密钥作为该组的正确密钥。使用卡方检验（**Chi-squared Test**）。哪个密钥值 k 使得解密后的文本片段最像英文（即卡方值最低），那个 k 就是正确的 $key[i]$ 。

```
# -*- coding: utf-8 -*-
import sys

# --- 全局常量 ---

# 加密时使用的字符集
TEXT_LIST = ' !"#%&\'()*+,-./0123456789:;<=>?
@ABCDEFGHIJKLMNPOQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~\t\n'
# 字符集长度，也是模运算的模数
MOD = len(TEXT_LIST)
# 字符到索引的快速查找映射
TEXT_MAP = {char: i for i, char in enumerate(TEXT_LIST)}
# 已知的密钥长度
KEY_LENGTH = 29

# 标准英文文本中，各字符的近似出现频率（概率）
# 这个列表有 97 个元素，与 TEXT_LIST 的长度完全匹配
ENGLISH_FREQS = [
    0.1828, 0.0001, 0.0001, 0.0001, 0.0001, 0.0001, 0.0001, 0.0001, 0.0001,
    0.0001, 0.0001, 0.0010, 0.0020, 0.0010, 0.0080, 0.0001,
    0.0040, 0.0040, 0.0040, 0.0040, 0.0040, 0.0040, 0.0040, 0.0040, 0.0040,
    0.0040, 0.0010, 0.0010, 0.0010, 0.0010, 0.0010, 0.0010, 0.0010,
    0.0651, 0.0125, 0.0221, 0.0321, 0.1031, 0.0208, 0.0152, 0.0468, 0.0599,
    0.0010, 0.0055, 0.0324, 0.0205, 0.0602, 0.0799, 0.0150,
    0.0008, 0.0496, 0.0550, 0.0751, 0.0225, 0.0080, 0.0173, 0.0012, 0.0154,
    0.0014,
    0.0001, 0.0001, 0.0001, 0.0001, 0.0001, 0.0001,
    0.0651, 0.0125, 0.0221, 0.0321, 0.1031, 0.0208, 0.0152, 0.0468, 0.0599,
    0.0010, 0.0055, 0.0324, 0.0205, 0.0602, 0.0799, 0.0150,
    0.0008, 0.0496, 0.0550, 0.0751, 0.0225, 0.0080, 0.0173, 0.0012, 0.0154,
    0.0014,
    0.0001, 0.0001, 0.0001, 0.0001, 0.0001, 0.0001
```

```
]
```

```
# ---- 辅助函数 ----
```

```
def calculate_chi_squared(text: str) -> float:
    """计算给定文本的卡方值，以判断其与英文的相似度。值越低，越像英文。"""
    text_len = len(text)
    if text_len == 0:
        return float('inf')

    observed_counts = [0] * MOD
    for char in text:
        if char in TEXT_MAP:
            observed_counts[TEXT_MAP[char]] += 1

    chi_squared_val = 0.0
    for i in range(MOD):
        expected_count = text_len * ENGLISH_FREQS[i]
        if expected_count > 0:
            difference = observed_counts[i] - expected_count
            chi_squared_val += difference**2 / expected_count
    return chi_squared_val
```

```
def decrypt_full(ciphertext: str, key: list) -> str:
    """使用给定的完整密钥列表解密全文。"""
    plain_text = []
    key_len = len(key)
    for i, char_c in enumerate(ciphertext):
        try:
            key_val = key[i % key_len]
            inv_k = pow(key_val, MOD - 2, MOD)
            c_idx = TEXT_MAP[char_c]
            p_idx = (c_idx * inv_k) % MOD
            plain_text.append(TEXT_LIST[p_idx])
        except (KeyError, ValueError):
            plain_text.append('?')
    return "".join(plain_text)
```

```
# ---- 主程序 ----
```

```
def main():
    """主执行函数，系统性破解密钥并解密。"""
    try:
        with open('cipher.txt', 'r', encoding='utf-8') as f:
            ciphertext = f.read().replace('\u00A0', ' ')
```

```

    print("成功从 'cipher.txt' 文件加载密文。")
except FileNotFoundError:
    print("错误: 'cipher.txt' 文件未在当前目录中找到。")
    sys.exit(1)

print(f"\n开始破解, 已知密钥长度为 {KEY_LENGTH}...")
found_key = []

# 1. 遍历密钥的每一个位置
for i in range(KEY_LENGTH):
    print(f"---- 正在破解密钥的第 {i+1}/{KEY_LENGTH} 位 ----")

    # 2. 提取出由该位置密钥加密的所有字符 (构成一个分组)
    sub_cipher = ciphertext[i::KEY_LENGTH]

    best_k_for_pos = -1
    min_chi_score = float('inf')

    # 3. 遍历所有可能的密钥值 (1 到 96)
    for k_guess in range(1, MOD):
        # 仅用当前的猜测值 k_guess 解密这个分组
        inv_k = pow(k_guess, MOD - 2, MOD)
        decrypted_sub_list = []
        for char_c in sub_cipher:
            if char_c in TEXT_MAP:
                c_idx = TEXT_MAP[char_c]
                p_idx = (c_idx * inv_k) % MOD
                decrypted_sub_list.append(TEXT_LIST[p_idx])
        decrypted_sub = "".join(decrypted_sub_list)

        # 4. 计算卡方值, 找到分数最低 (最像英文) 的那个密钥值
        score = calculate_chi_squared(decrypted_sub)
        if score < min_chi_score:
            min_chi_score = score
            best_k_for_pos = k_guess

    print(f"找到第 {i+1} 位密钥: {best_k_for_pos} (卡方值: {min_chi_score:.2f})")
    found_key.append(best_k_for_pos)

# 5. 组合密钥并解密全文
print("\n" + "="*50)
print("密钥破解完成!")
print(f"破解出的完整密钥为: {found_key}")
print("="*50 + "\n")

```

```

full_plain_text = decrypt_full(ciphertext, found_key)

print("---- 完整解密后的明文如下 ----")
print(full_plain_text)

# 6. 查找并高亮显示 flag
flag_start = full_plain_text.find("flag{")
if flag_start != -1:
    flag_end = full_plain_text.find("}", flag_start)
    if flag_end != -1:
        print("\n" + "="*50)
        print("---- 成功找到 Flag ----")
        print(full_plain_text[flag_start : flag_end + 1])
        print("="*50)

if __name__ == "__main__":
    main()

```

```

=====
密钥破解完成！
破解出的完整密钥为：[51, 14, 22, 90, 67, 43, 17, 82, 48, 17, 41, 33, 89, 94, 60, 80, 13, 72, 79, 18, 65, 50, 20, 6, 36, 39, 56, 7, 65]
=====

---- 完整解密后的明文如下 ----
By the mid-nineteenth century, the term "icebox" had entered the American language, but ice was still only beginning to affect the diet of ordinary citizens in the United States. The ice trade grew with the growth of cities. Ice was used in hotels, taverns, and hospitals, and by some forward-looking city dealers in fresh meat, fresh fish, and butter. After the Civil War (1861-1865), as ice was used to refrigerate freight cars, it also came into household use. Even before 1880, half the ice sold in New York, Philadelphia, and Baltimore, and one-third of that sold in Boston and Chicago, went to families for their own use. This had become possible because a new household convenience, the icebox, a precursor of the modern refrigerator, had been invented. Making an efficient icebox was not as easy as we might now suppose. In the early nineteenth century, the knowledge of the physics of heat, which was essential to a science of refrigeration, was rudimentary. The commonsense notion that the best icebox was one that prevented the ice from melting was of course mistaken, for it was the melting of the ice that performed the cooling. Nevertheless, early efforts to economize ice included wrapping the ice in blankets, which kept the ice from doing its job. Not until near the end of the nineteenth century did inventors achieve the delicate balance of insulation and circulation needed for an efficient icebox.
fLaG:AAA{i_like_T0ef1_v3ry_M3uh!!!!}
But as early as 1803, an ingenious Maryland farmer, Thomas Moore, had been on the right track. He owned a farm about twenty miles outside the city of Washington, for which the village of Georgetown was the market center. When he used an icebox of his own design to transport his butter to market, he found that customers would pass up the rapidly melting stuff in the tubs of his competitors to pay a premium price for his butter, still fresh and hard in neat, one-pound bricks. One advantage of his icebox, Moore explained, was that farmers would no longer have to travel to market at night in order to keep their produce cool. Perhaps the most obvious way artistic creation reflects how people live is by mirroring the environment - the materials and technologies available to a culture. Stone, wood, tree bark, clay, and sand are generally available materials. In addition, depending on the locality, other resources may be accessible: shells, horns, gold, copper, and silver. The different uses to which societies put these materials are of interest to anthropologists who may ask, for example, why people choose to use clay and not copper when both items are available. Although there are no conclusive answers yet, the way in which a society views its environment is sometimes apparent in its choice and use of artistic materials. The use of certain metals, for example, may be reserved for ceremonial objects of special importance. Or the belief in the supernatural powers of a stone or tree may cause a sculptor to be sensitive to that material. What is particularly meaningful to anthropologist is the realization that although the materials available to a society may to some extent limit or influence what it can do artistically, the materials by no means determine what is done. Why do the artists in Japanese society rake sand into patterns; and the artists in Roman society melt sand to form glass? Moreover, even when the same material is used in the same way by members of different societies, the form or style of the work varies enormously from culture to culture. A society may simply choose to represent objects or phenomena that are important to its population. An examination of the art of the Middle Ages tells us something about the medieval preoccupation with theological doctrine. In addition to revealing the primary conce

```

fLaG:AAA{i_like_T0ef1_v3ry_M3uh!!!!}

本次实验最精彩的部分，无疑是卡方检验在确定密钥值时展现出的惊人效果。它的原理，本质上是将一个看似主观的判断（“这段文字像不像英文？”）变成了一个可以量化的、精确的数学问题。

$$\chi^2 = \sum \frac{(\text{观测次数} - \text{期望次数})^2}{\text{期望次数}}$$

1. (观测次数 - 期望次数)：对于每一个字母（比如'A'），看看我们解密的文本里出现了多少次，再看看标准英文里期望它出现多少次，然后算个差值。
2. (...)^2：把这个差值平方。这样做有两个目的：一是让结果总是正数，二是可以放大那些差距大的项，让“错误”更明显。

3. / 期望次数：再除以期望次数。这是为了“标准化”。比如，对于罕见的'Z'，差2个可能就很重要；但对于常见的'E'，差2个可能只是正常波动。
4. Σ (求和)：最后，把我们字符表里所有字符（A-Z, a-z, 空格, 符号等）的这个计算结果全部加起来，得到一个总的“差异度分数”，也就是卡方值。

challenge HCP

sage复现

希尔密码的破解本质上是一个线性代数问题。其核心是利用已知的明文-密文对，来求解一个未知的线性变换（即密钥矩阵）。

任务 1: 创建随机可逆矩阵并求逆 (10分)

1. 定义数学模型

首先，我们定义所有相关的数学对象。所有的运算都在整数模256环中进行，我们记为 Z_{256} 。

- 密钥矩阵 (Key Matrix):

令密钥矩阵为 K 。根据题目，这是一个 3×3 的方阵，其所有元素 $k_{ij} \in Z_{256}$ 。

$$K = \begin{pmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{pmatrix}$$

在题目中，这个矩阵被称为 MT。

- 明文矩阵 (Plaintext Matrix):

令完整的明文矩阵为 P 。这是一个 3×10 的矩阵，由 flag 字符串的ASCII码值按列填充而成。

$$P = \begin{pmatrix} p_{1,1} & p_{1,2} & \cdots & p_{1,10} \\ p_{2,1} & p_{2,2} & \cdots & p_{2,10} \\ p_{3,1} & p_{3,2} & \cdots & p_{3,10} \end{pmatrix}$$

其中 $p_{i,j} = \text{ord}(\text{flag}[(i-1) + (j-1) \times 3]) \pmod{256}$ 。在题目中，这个矩阵被称为 FT。

- 密文矩阵 (Ciphertext Matrix):

令完整的密文矩阵为 C 。这是一个 3×10 的矩阵。

$$C = \begin{pmatrix} c_{1,1} & c_{1,2} & \cdots & c_{1,10} \\ c_{2,1} & c_{2,2} & \cdots & c_{2,10} \\ c_{3,1} & c_{3,2} & \cdots & c_{3,10} \end{pmatrix}$$

在题目中，这个矩阵被称为 RT。

- 加密方程 (Encryption Equation):
希尔密码的加密过程是矩阵乘法。整个加密过程可以表示为:

$$C = K \cdot P$$

这个方程在 Z_{256} 上成立。

目标： 随机生成一个在模256下可逆的3x3矩阵 `MT`，然后计算它的逆矩阵 `MT_inv`。

说明： 一个矩阵 `A` 在模 `n` 下可逆，当且仅当其行列式 $|A|$ 与 `n` 互质。对于本题的模 `n=256=28` 来说，这意味着 $|A|$ 必须是一个奇数。

```
# 定义在模256下的整数环
Z256 = Zmod(256)

# 随机生成一个 3x3 矩阵 MT
# 为了确保它可逆，我们循环生成，直到找到一个可逆的为止
while True:
    # 从Z256中随机选取9个元素，构成一个3x3矩阵
    MT = random_matrix(Z256, 3, 3)
    # is_invertible() 会检查 det(MT) 是否与 256 互质
    if MT.is_invertible():
        break

# 求 MT 的逆矩阵
MT_inv = MT.inverse()

# 打印结果
print("随机生成的矩阵 MT:")
print(MT)
print("\nMT 的行列式 :")
print(MT.det())
print("\nMT 的逆矩阵 MT_inv:")
print(MT_inv)
print("\n验证 MT * MT_inv 是否为单位矩阵:")
print(MT * MT_inv)
```


随机生成的矩阵 MT:

```
[199  22 230]
[ 91  89 134]
[ 11  40  91]
```

MT 的行列式 :

233

MT 的逆矩阵 MT_inv:

```
[ 59 110  94]
[129  35 216]
[ 29 202  85]
```

验证 $MT * MT_inv$ 是否为单位矩阵:

```
[1 0 0]
[0 1 0]
[0 0 1]
```

因为 $C = K \cdot P$, 所以

$$P = K^{-1} \cdot C$$

任务2:随机设置 flag 生成 FT, 计算 RT, 再通过 RT 和 MT 求出 FT 的值, 与原 FT 进行比对 (10分)

写代码求解。

```
# 假设我们已经有了上一部分生成的 MT 和 MT_inv

# 1. 随机设置一个 flag (长度为30的倍数)
my_flag = "flag{this_is_Asecret_sagemath}" # 恰好30个字符
print(f"原始 Flag: {my_flag}\n")

# 2. 将 flag 转换为明文矩阵 FT (3x10)
FT = matrix(Z256, 3, 10)
for i in range(3):
    for j in range(10):
```

```
# 按列填充
FT[i, j] = ord(my_flag[i + j * 3])

print("明文矩阵 FT:")
print(FT)

# 3. 使用 MT 加密 FT, 得到密文矩阵 RT
RT = MT * FT
print("\n加密后的密文矩阵 RT:")
print(RT)

# 4. (核心) 使用 MT_inv 和 RT 来恢复 FT
FT_recovered = MT_inv * RT
print("\n使用逆矩阵恢复的明文矩阵 FT_recovered:")
print(FT_recovered)

# 5. 将恢复的矩阵 FT_recovered 转换回字符串, 进行比对
recovered_flag = ""
for j in range(10):
    for i in range(3):
        recovered_flag += chr(int(FT_recovered[i, j]))

print(f"\n恢复的 Flag: {recovered_flag}\n")
assert my_flag == recovered_flag
print("比对成功! 原始 Flag 与恢复的 Flag 完全一致。")
```

随机生成的矩阵 MT:

```
[ 26 247 239]
[ 38 125 248]
[167  63  50]
```

MT 的行列式 :
249

MT 的逆矩阵 MT_inv:

```
[242 235 125]
[ 60  67  26]
[  7  83  72]
```

验证 $MT * MT_inv$ 是否为单位矩阵:

```
[1 0 0]
[0 1 0]
[0 0 1]
```

原始 Flag: flag{this_is_Asecret_sagemath}

明文矩阵 FT:

```
[102 103 104  95  95 101 101 115 101 116]
[108 123 105 105  65  99 116  97 109 104]
[ 97 116 115 115 115 114  95 103  97 125]
```

加密后的密文矩阵 RT:

```
[ 31 111  60  82 186  53 223 110 252 211]
[216 185  29 199  63 197 170  55  47  24]
[ 16  30  37  70 110 132 253  2 168 174]
```

使用逆矩阵恢复的明文矩阵 FT_recovered:

```
[102 103 104  95  95 101 101 115 101 116]
[108 123 105 105  65  99 116  97 109 104]
[ 97 116 115 115 115 114  95 103  97 125]
```

恢复的 Flag: flag{this_is_Asecret_sagemath}

比对成功! 原始 Flag 与恢复的 Flag 完全一致。

HSC：已知明文攻击的原理推导

我们的目标是求解未知的密钥矩阵 K 。已知明文攻击的核心思想是，我们已经知道了 P 的一部分和 C 的相应部分。

构造子矩阵：

假设我们已知 flag 的前9个字符。这9个字符可以构成明文矩阵 P 的前3列。我们称这个 3×3 的明文子矩阵为 P_{sub} 。

$$P_{sub} = \begin{pmatrix} p_{1,1} & p_{1,2} & p_{1,3} \\ p_{2,1} & p_{2,2} & p_{2,3} \\ p_{3,1} & p_{3,2} & p_{3,3} \end{pmatrix}$$

相应地，这部分明文被加密后，会得到密文矩阵 C 的前3列。我们称这个 3×3 的密文子矩阵为 C_{sub} 。

$$C_{sub} = \begin{pmatrix} c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,1} & c_{3,2} & c_{3,3} \end{pmatrix}$$

2. 建立方程组：

根据加密总方程 $C = K \cdot P$ ，我们可以得到关于这两个子矩阵的方程：

$$C_{sub} = K \cdot P_{sub}$$

求解密钥矩阵 K ：

这是一个矩阵方程，我们的目标是解出 K 。为了从方程右侧“消去” P_{sub} ，我们需要用到它的逆矩阵，记为 P_{sub}^{-1} 。

对方程两边右乘 P_{sub}^{-1} ： $C_{sub} \cdot P_{sub}^{-1} = (K \cdot P_{sub}) \cdot P_{sub}^{-1}$

根据矩阵乘法的结合律，我们可以重新组合括号： $C_{sub} \cdot P_{sub}^{-1} = K \cdot (P_{sub} \cdot P_{sub}^{-1}) = K$

3. 矩阵可逆性的数学条件

上述推导能够成立，其充要条件是明文子矩阵 P_{sub} 在 Z_{256} 上是可逆的。

一个方阵 A 在模 n 的环上可逆，当且仅当其行列式 $\det(A)$ 与模数 n 互质。即：

$$\gcd(\det(A), n) = 1$$

在本题中，模数 $n = 256 = 2^8$ 。因此，矩阵 P_{sub} 可逆的条件是：

$$\gcd(\det(P_{sub}), 256) = 1$$

由于 256 的唯一质因数是 2，这个条件等价于 $\det(P_{sub})$ 不能被 2 整除，即 $\det(P_{sub})$ 必须是一个奇数。

这就是为什么我们需要猜测并找到一个能够构成行列式为奇数的 3×3 明文矩阵的 plaintext

fragment。如果我们的猜测（例如 "flag{abc}"）使得 $\det(P_{sub})$ 是一个偶数，那么 P_{sub} 在模 256 下不可逆，我们就无法用它来求解 K ，必须尝试其他的明文片段。

4. 一旦通过上述方法成功求出了密钥矩阵 K ，我们就可以用它来解密整个密文矩阵 C 。

在此题中，

先提取 RT（即上面的 C ）

```
# SageMath 环境

import string

# 定义在模256下的整数环和向量空间
Z256 = Zmod(256)
V = Z256^3

# 检查一个数值是否是可打印ASCII码
def is_printable(char_code):
    # Printable ASCII characters are in the range [32, 126]
    return 32 <= char_code <= 126

# 1. 已知密文
result =
b'\xfc\xf2\x1dE\xf7\xd8\xf7\x1e\xed\xccQ\x8b9:z\xb5\xc7\xca\xea\xcd\xb4b\xdd
\xcb\xf2\x939\x0b\xec\xf2'

# 2. 按列填充，正确构建 RT 矩阵
RT = matrix(Z256, 3, 10)
for j in range(10): # 遍历列
    for i in range(3): # 遍历行
        RT[i, j] = result[i + j * 3]

print("正确构建的密文矩阵 RT (3x10):")
print(RT)
```

正确构建的密文矩阵 RT (3x10):

[252	69	247	204	57	181	234	98	242	11]
[242	247	30	81	58	199	205	221	147	236]
[29	216	237	139	122	202	180	203	57	242]

约束 3.1: 部分已知的明文 P 的元素 根据 `flag = "AAA{????????????????????????????}"`，我们可以确定5个明文矩阵 P 中的元素值：

- `flag[0] = 'A'  $\Rightarrow p_{1,1} = \text{ord}('A') = 65$`
- `flag[1] = 'A'  $\Rightarrow p_{2,1} = \text{ord}('A') = 65$`
- `flag[2] = 'A'  $\Rightarrow p_{3,1} = \text{ord}('A') = 65$`
- `flag[3] = '{'  $p_{1,2} = \text{ord}('{') = 123$`
- `flag[29] = '}'  $p_{3,10} = \text{ord}('{') = 123$`

约束 3.2: 未知明文的字符集 flag 中所有未知字符 ? 被描述为 `printable char`。这意味着它们对应的ASCII码值必须在一个有限且明确的范围内。标准的英文可打印字符范围是 `[32, 126]`。这是一个极其强大的约束，可以将爆破的验证空间从256缩小到95。对于所有未知的 $p_{i,j}$ ，我们有：

$$32 \leq p_{i,j} \leq 126$$

约束 3.3: 密钥的可逆性 代码中的 `assert MT.is_invertible()` 保证了密钥矩阵 K 是可逆的。这意味着 $\det(K)$ 与 256 互质，即 $\det(K)$ 是一个奇数。这也保证了解密密钥矩阵 $D = K^{-1}$ 存在且同样可逆。

3. 基于约束建立的线性方程组

我们将利用解密方程 $P = D \cdot C$ 和5个已知明文点，建立一个关于解密密钥矩阵 D 元素 (d_{ij}) 的线性方程组。令 D 的第 i 行向量为 $d_i = (d_{i1}, d_{i2}, d_{i3})$, C 的第 j 列向量为 c_j 。则 $p_{i,j} = d_i \cdot c_j$ 。

- 方程 1 ($p_{1,1}$): $d_1 \cdot c_1 = 65 \Rightarrow 252d_{11} + 242d_{12} + 29d_{13} \equiv 65 \pmod{256}$
- 方程 2 ($p_{2,1}$): $d_2 \cdot c_1 = 65 \Rightarrow 252d_{21} + 242d_{22} + 29d_{23} \equiv 65 \pmod{256}$
- 方程 3 ($p_{3,1}$): $d_3 \cdot c_1 = 65 \Rightarrow 252d_{31} + 242d_{32} + 29d_{33} \equiv 65 \pmod{256}$
- 方程 4 ($p_{1,2}$): $d_1 \cdot c_2 = 123 \Rightarrow 69d_{11} + 247d_{12} + 216d_{13} \equiv 123 \pmod{256}$
- 方程 5 ($p_{3,10}$): $d_3 \cdot c_{10} = 125 \Rightarrow 11d_{31} + 236d_{32} + 242d_{33} \equiv 125 \pmod{256}$

4. 求解策略与实现

我们所建立的线性方程组包含 9 个未知变量（解密矩阵 D 的九个元素），但仅有 5 个独立的线性方程。这是一个典型的欠定方程组 (**underdetermined system**)，无法直接通过代数方法求得唯一解。因此，我们必须引入问题中的其他关键约束来筛选并找到正确解。

本实验的求解核心思想是，综合利用已知的三个强约束条件，通过一种“逐行击破，组合筛选”的策略，逐步缩小解空间，最终恢复出唯一的解密密钥 D。

4.1. 求解策略：逐行暴力搜索与约束验证

我们将解密矩阵 D 的求解分解为三个独立的部分，即逐行求解 D 的三个行向量 d_1, d_2, d_3 。这样做是可行的，因为我们建立的 5 个方程天然地按行分离。

1. $d_1=(d_{11},d_{12},d_{13})$ 的求解：依赖于方程 1 和方程 4。
2. $d_2=(d_{21},d_{22},d_{23})$ 的求解：依赖于方程 2。
3. $d_3=(d_{31},d_{32},d_{33})$ 的求解：依赖于方程 3 和方程 5。

对于每一行，我们采用一种结合了暴力搜索和约束验证的算法：

1. **识别自由变量**：确定每行求解中自由变量的个数。例如，求解 d_1 时有 3 个未知数和 2 个方程，存在自由变量。为了保证搜索的完备性，我们设定两个变量为自由变量进行遍历。
2. **暴力搜索**：遍历所有自由变量的可能取值（在 $\mathbb{Z}_{256}$ 环中，即 0 到 255）。
3. **求解与验证**：对于自由变量的每一个取值组合，利用其中一个线性方程计算出剩余变量的值。然后，将求得的完整行向量代入该行的另一个线性方程（如果存在）进行验证，确保其满足所有代数约束。
4. **可打印性约束验证**：将通过代数验证的候选行向量 d_i 与完整的密文矩阵 RT 相乘，得到一行解密后的明文候选项。检查这一整行的 10 个字符的 ASCII 值是否全部落在可打印范围 $[32, 126]$ 内。只有满足此条件的行向量，才被视为一个有效的**候选行**。

4.2. 最终筛选：基于可逆性约束的组合

上述步骤可能会为某一行找到多个满足条件的候选行向量。实验代码的输出清晰地证明了这一点（第二行找到了 5 个候选选项）。此时，就需要利用最后一个，也是最关键的约束：`assert MT.is_invertible()`。

此约束意味着解密矩阵 $D = MT^{-1}$ 本身也必须是可逆的。在 $\mathbb{Z}_{256}$ 上，一个矩阵可逆的充要条件是其行列式 $\det(D)$ 与 256 互质，即 $\det(D)$ 必须为奇数。

我们的最终筛选策略如下：

1. 收集所有候选行：执行 4.1 中的算法，收集每一行的所有有效候选行向量。
2. 遍历组合：使用笛卡尔积的方式，遍历所有候选行向量的可能组合，构成一个完整的 3×3 候选解密矩阵 $D_{candidate}$ 。
3. 检验行列式：计算每个 $D_{candidate}$ 的行列式。如果 $\det(D_{candidate})$ 为奇数，则我们找到了唯一正确的解密矩阵 D 。

编写代码求解如下

```
# SageMath 环境

# 0. 基础设置
Z256 = Zmod(256)
```

```

def is_printable(char_code):
    """检查一个数值是否是可打印ASCII码 (范围 [32, 126])"""
    return 32 <= char_code <= 126

# 1. 已知密文
result =
b'\xfc\xf2\xdE\xf7\xd8\xf7\xe\xed\xccQ\x8b9:z\xb5\xc7\xca\xea\xcd\xb4b\xdd
\xcb\xf2\x939\x0b\xec\xf2'

# 2. 按列填充, 正确构建密文矩阵 RT
RT = matrix(Z256, 3, 10)
for j in range(10):      # 遍历列
    for i in range(3):    # 遍历行
        RT[i, j] = result[i + j * 3]

print("✅ 已成功构建密文矩阵 RT (3x10)")
print("-" * 40)

# 提取方程中需要的密文列向量
c1, c2, c10 = RT.column(0), RT.column(1), RT.column(9)

# 存储所有可能的候选行
d1_candidates, d2_candidates, d3_candidates = [], [], []

# -----
# Part 3.1: 搜索D的第一行所有可能的候选项
# -----
print("🕒 搜索 D 的第一行所有候选项...")
inv69 = Z256(69)^(-1)
for d12_val in range(256):
    for d13_val in range(256):
        # 根据方程 d1*c2 = 123 求解 d11
        d11_sol = (123 - c2[1]*d12_val - c2[2]*d13_val) * inv69
        # 验证是否满足另一个方程 d1*c1 = 65
        if c1[0]*d11_sol + c1[1]*d12_val + c1[2]*d13_val == 65:
            d1_candidate = vector(Z256, [d11_sol, d12_val, d13_val])
            # 验证解密出的整行明文是否都可打印
            if all(is_printable(p) for p in d1_candidate * RT):
                d1_candidates.append(d1_candidate)
print(f"🕒 找到 {len(d1_candidates)} 个第一行候选项: {d1_candidates}")
print("-" * 40)

# -----
# Part 3.2: 搜索D的第二行所有可能的候选项
# -----
print("🕒 搜索 D 的第二行所有候选项...")

```



```

inv29 = Z256(29)^(-1)
for d21_val in range(256):
    for d22_val in range(256):
        # 根据方程  $d2 * c1 = 65$  求解  $d23$ 
        d23_sol = (65 - c1[0]*d21_val - c1[1]*d22_val) * inv29
        d2_candidate = vector(Z256, [d21_val, d22_val, d23_sol])
        # 验证解密出的整行明文是否都可打印
        if all(is_printable(p) for p in d2_candidate * RT):
            d2_candidates.append(d2_candidate)
print(f"🔍 找到 {len(d2_candidates)} 个第二行候选项: {d2_candidates}")
print("-" * 40)

# -----
# Part 3.3: 搜索D的第三行所有可能的候选项
# -----

print("🕒 搜索 D 的第三行所有候选项...")
inv11 = Z256(11)^(-1)
for d32_val in range(256):
    for d33_val in range(256):
        # 根据方程  $d3 * c10 = 125$  求解  $d31$ 
        d31_sol = (125 - c10[1]*d32_val - c10[2]*d33_val) * inv11
        # 验证是否满足另一个方程  $d3 * c1 = 65$ 
        if c1[0]*d31_sol + c1[1]*d32_val + c1[2]*d33_val == 65:
            d3_candidate = vector(Z256, [d31_sol, d32_val, d33_val])
            # 验证解密出的整行明文是否都可打印
            if all(is_printable(p) for p in d3_candidate * RT):
                d3_candidates.append(d3_candidate)
print(f"🔍 找到 {len(d3_candidates)} 个第三行候选项: {d3_candidates}")
print("-" * 40)

# -----
# Part 4: 组合所有候选项, 找到唯一可逆的解密矩阵 D
# -----

print("🕒 组合所有候选项, 寻找唯一可逆的解密矩阵...")
D_sol = None
for d1 in d1_candidates:
    for d2 in d2_candidates:
        for d3 in d3_candidates:
            D_candidate = matrix([d1, d2, d3])
            # is_invertible() 在  $Z_{mod}(n)$  中等价于检查  $\det()$  是否与  $n$  互质 (即是否为奇数)
            if D_candidate.is_invertible():
                D_sol = D_candidate
                print(f"✅ 找到唯一可逆解密矩阵!  $\text{Det}(D) = \{D\_sol.\det()\}$  (奇数)")
                break
if D_sol: break

```

```

        if D_sol: break

# 5. 输出所有最终结果
if D_sol:
    # 原始加密密钥矩阵 MT
    MT_sol = D_sol.inverse()

    # 明文矩阵 P (FT)
    FT_sol = D_sol * RT

    # 还原 Flag 字符串
    flag_chars = [''] * 30
    for j in range(10):
        for i in range(3):
            flag_chars[i + j * 3] = chr(int(FT_sol[i, j]))
    final_flag = "".join(flag_chars)

    # 格式化输出
    print("\n" + "="*50)
    print("🎉 所有计算已完成, 最终结果如下: ")
    print("="*50 + "\n")

    print("🚩 Flag:")
    print(final_flag)
    print("\n-----\n")

    print("明文矩阵 P (FT):")
    print(FT_sol)
    print("\n-----\n")

    print("解密密钥矩阵 D (MT.inverse()):")
    print(D_sol)
    print("\n-----\n")

    print("原始加密密钥矩阵 MT:")
    print(MT_sol)
else:
    print("\n❌ 错误: 未能从候选项中找到任何可逆的解密矩阵。")

```

✅ 已成功构建密文矩阵 RT (3x10)

🕒 搜索 D 的第一行所有候选项...

🏆 找到 1 个第一行候选项: [(124, 177, 43)]

🕒 搜索 D 的第二行所有候选项...

🏆 找到 5 个第二行候选项: [(16, 51, 135), (124, 177, 43), (137, 33, 143), (157, 38, 157), (208, 21, 147)]

🕒 搜索 D 的第三行所有候选项...

🏆 找到 1 个第三行候选项: [(137, 33, 143)]

🕒 组合所有候选项, 寻找唯一可逆的解密矩阵...

✅ 找到唯一可逆解密矩阵! Det(D) = 11 (奇数)

🏆 所有计算已完成, 最终结果如下:

🚩 Flag:

AAA{s11Hp*aB4c!1mZQT3^gTn1L&1}

明文矩阵 P (FT):

```
[ 65 123 49 42 52 49 81 94 110 38]
[ 65 115 72 97 99 109 84 103 49 49]
[ 65 108 112 66 33 90 51 84 76 125]
```

解密密钥矩阵 D (MT.inverse()):

```
[124 177 43]
[157 38 157]
[137 33 143]
```

原始加密密钥矩阵 MT:

```
[ 23 132 97]
[ 54 99 217]
[ 21 87 113]
```

借用llm编写的prompt就是复制了自己写的约束3.1-第四节之前的所有内容, 然后命令ai暴力求解。

Challenge RSA

-----BEGIN RSA PRIVATE KEY-----

MIGrAgEAAiEAwmNq5cPY5D/7l6sJAo8arGwL9s09c0vKKBv/6X++MN0CAwEAAQIgGAZ5m9RM5kkS
K3i0MGDHHvi3f7FZPghC2gY...

第一部分: 解析DER格式的RSA密钥 (30分)

PEM格式的密钥本质上是DER格式密钥的Base64编码。我们的第一步是解码给定的PEM内容, 然后解析其ASN.1 DER结构。

1. Base64解码

给定的PEM内容是：

```
MIGrAgEAAiEAWmNq5cPY5D/7l6sJAo8arGwL9s09c0vKKBv/6X++MN0CAwEAAQIgGAZ5m9RM5kkSK3i
0MGDHHvi3f7FZPghC2gY...
```

我们去掉 ...，得到Base64字符串：

```
MIGrAgEAAiEAWmNq5cPY5D/7l6sJAo8arGwL9s09c0vKKBv/6X++MN0CAwEAAQIgGAZ5m9RM5kkSK3i
0MGDHHvi3f7FZPghC2gY= (为了使其成为有效的Base64，我补上了一个结尾的'=,)
```

使用Base64解码工具，可以得到其十六进制表示的DER数据：

分组

```
30 81 AB 02 01 00 02 21 00 C2 63 6A E5 C3 D8 E4 3F FB 97 AB 09 02 8F 1A AC 6C
0B F6 CD 3D 70 EB CA 28 1B FF E9 7F BE 30 DD 02 03 01 00 01 02 20 18 06 79 9B
D4 4C E6 49 12 2B 78 B4 30 60 C7 86 F8 B7 7F B1 59 3E 08 42 DA 06
```

2. ASN.1 DER格式解析

ASN.1是一种数据描述语言，DER是其一种编码规则。其结构是TLV (Tag-Length-Value)。

- **Tag:** 标识数据类型 (例如 0x30 代表 SEQUENCE，0x02 代表 INTEGER)。
- **Length:** 标识Value的长度。
- **Value:** 实际的数据。

根据PKCS#1 (RFC 8017) 标准，RSA私钥的ASN.1结构是一个 SEQUENCE，包含以下字段：

```
RSAPrivateKey ::= SEQUENCE { version Version, modulus INTEGER, -- n
publicExponent INTEGER, -- e privateExponent INTEGER, -- d prime1 INTEGER, -- p
prime2 INTEGER, -- q exponent1 INTEGER, -- d mod (p-1) exponent2 INTEGER, -- d
mod (q-1) coefficient INTEGER -- (inverse of q) mod p }
```

现在我们来手动解析上面的十六进制DER数据：

分组即

```
3081ab020100022100c2636ae5c3d8e43ffb97ab09028f1aac6c0bf6cd3d70ebca281bffe97fbe30
dd020301000102201806799bd44ce649122b78b43060c786f8b77fb1593e0842da06
```

```
3081ab020100022100
```

```
c2636ae5c3d8e43ffb97ab09028f1aac6c0bf6cd3d70ebca281bffe97fbe30dd
```

```
0203 010001
```

```
0220 1806799bd44ce649122b78b43060c786f8b77fb1593e0842da06
```

偏移量	Tag	Length	Value	字段含义
0x00	30	81 AB	(接下来的171字节)	SEQUENCE: 整个密钥结构的开始。81 表示长度用1个字节表示，即 AB (171)。
0x03	02	01	00	INTEGER: version 。版本号为0，代表双素数 (p,q) 的RSA。
0x05	02	21	00 C2 63...30 DD (33字节)	INTEGER: modulus (n) 。模数n。开头的 00 是为了确保该数被解释为正数。
0x28	02	03	01 00 01	INTEGER: publicExponent (e) 。公钥指数e，通常是65537 (0x010001)。
0x2D	02	20	18 06 79...DA 06 (32字节)	INTEGER: privateExponent (d) 。私钥指数d，但这里的长度只有32字节 (256位)，而n是33字节 (264位)，说明d是不完整的！这就是问题的关键。
0x4F	-	-	(数据在此处被截断)	缺失的数据: prime1 (p), prime2 (q), exponent1, exponent2, coefficient 都丢失了。

字段值提取:

- **n (十六进制):**
c2636ae5c3d8e43ffb97ab09028f1aac6c0bf6cd3d70ebca281bffe97fbe30dd
- **e (十进制):** 65537
- **d_partial (d的高位部分, 十六进制):**
1806799bd44ce649122b78b43060c786f8b77fb1593e0842da06... (后面是未知的低位)

第二部分：使用factordb分解模数 (10分)

这是一个标准的检查步骤。如果模数 n 比较简单，或者是由一些小素数构成，那么可以直接在 factordb 等在线工具中分解。

步骤:

1. 将我们从第一步中得到的模数 n 的十六进制转换为十进制。 n_hex = c2636ae5c3d8e43ffb97ab09028f1aac6c0bf6cd3d70ebca281bffe97fbe30dd n_dec = 87924348264132406875276140514499937145050893665602592992418171647042491658461
2. 访问 [FactorDB](#) 网站。

3. 将这个十进制数输入查询框。

搜索	序列	报告结果	因子表	地位	下载	API
87924348264132406875276140514499937145050893665602592992418171647042491658461 分解因数!						
结果:						
地位 (?)	数字	数字				
FF	77 (显示)	8792434826...61	$= 275127860351348928173285174381581152299_{<77>} \cdot 319576316814478949870590164193048041239_{<39>}$			

编写解密脚本

```
# SageMath 或装有 gmpy2 的 Python 环境均可运行
# --- 已知参数和新发现的因子 ---
# 1. 从 factordb 得到的核心因子 p 和 q
p = 275127860351348928173285174381581152299
q = 319576316814478949870590164193048041239
# 2. 题目中已知的 e 和 n
# 我们可以通过 p*q 重新计算 n 来验证
n = p * q
e = 65537
# 3. 题目中给出的十六进制密文 c
c_hex = "1c194cd4f48d77b2e14cace43869bea17615ab23da0ef63b7bf56116ad3ac93b"
c = int(c_hex, 16)
# --- 解密过程 ---
# 1. 计算 phi(n)
phi_n = (p - 1) * (q - 1)
print(f"[计算] 成功计算出 phi(n): {phi_n}")
# 2. 计算私钥 d
# pow(e, -1, phi_n) 是计算模反元素的标准方法 (需要 Python 3.8+)
d = pow(e, -1, phi_n)
print(f"[计算] 成功计算出完整私钥 d: {d}")
# 3. 解密密文 c
m = pow(c, d, n)
print(f"\n[解密] 成功解密出明文 (整数形式): {m}")
# 4. 将明文数字转换为可读的字符串
try:
    m_hex = hex(m)[2:]
    # 如果十六进制长度为奇数, 前面补0
    if len(m_hex) % 2 != 0:
        m_hex = '0' + m_hex
    flag_bytes = bytes.fromhex(m_hex)
    print(f"明文 (bytes 格式): {flag_bytes}")

    # CTF的flag通常是UTF-8编码的字符串
    final_flag = flag_bytes.decode('utf-8', errors='ignore')
    print(f"\n✅ 最终答案 (Flag): {final_flag}")
```

```
except Exception as ex:
    print(f"\n[错误] 将明文转换为字符串时出错: {ex}")
```

```
>>> quit()
(venv) kaisenye@kaisendeMacBook-Air challenge2.2 % python3 jiemi.py
[计算] 成功计算出 phi(n): 87924348264132406875276140514499937144456189488436765114374296308467862464924
[计算] 成功计算出完整私钥 d: 10866948760844599168252082612378495977388271279679231539839049698621994994673
[解密] 成功解密出明文 (整数形式): 29515632154944420859813428452243431312861742734353923499820016460871039536765
      明文 (bytes 格式): b'AAA{N3veR_Le4k_PR1va7eK3y_Ag41N}'
✓ 最终答案 (Flag): AAA{N3veR_Le4k_PR1va7eK3y_Ag41N}
```

AAA{N3veR_Le4k_PR1va7eK3y_Ag41N}

第三部分：RSA私钥高位攻击 (15分/35分)

这是本题的核心和难点。我们已知 n ， e 和私钥 d 的高位部分 (d_{partial})。我们需要恢复出 d 的完整值。这种攻击通常基于 **Coppersmith's Method** (科珀史密斯方法) 和格理论 (Lattice Theory)。

攻击原理详解 (35分)

- RSA的基本关系:** 我们知道 $e * d \equiv 1 \pmod{\phi(n)}$ 。其中 $\phi(n) = (p-1)(q-1) = n - (p+q) + 1$ 。这个同余式可以写成等式 $e * d - k * \phi(n) = 1$ ，其中 k 是一个未知的整数。
- 关键近似:** 由于 p 和 q 大约是 $\sqrt{n}$ 的量级，所以 $p+q$ 相比于 n 是一个非常小的值。因此， $\phi(n)$ 非常接近 n 。我们可以将 $\phi(n)$ 近似为 n ，得到 $e * d - k * n \approx 1$ 。
- 构建多项式:** 我们已知 d 的高位。假设 n 的位数为 L_n ，我们已知 d 的高 t 位。我们可以将完整的 d 表示为 $d = d_{\text{partial}} + x$ 。
 - d_{partial} 是我们已知的高位部分，可以精确计算出来 (例如， $d_{\text{partial}} = \text{known_hex} * 2^m$ ， m 是未知部分的位数)。
 - x 是我们未知的低位部分，它是一个相对较小的数。我们的目标就是求出 x 。

将 $d = d_{\text{partial}} + x$ 代入到 $e * d \equiv 1 \pmod{\phi(n)}$ 中，会非常复杂，因为 $\phi(n)$ 未知。一个更有效的方法是直接攻击 $e * d = 1 + k(n - (p+q) + 1)$ 这个关系。

一个更著名的攻击变体 (Boneh-Durfee攻击) 是研究 $e * d_0 - k(N - s_0 + 1) = 1$ ，其中 d_0 是 d 的低位， s_0 是 $p+q$ 的低位。

但对于本题，更直接的思路是利用 **Coppersmith's Short Pad Attack**。我们构建一个关于未知数 x 的多项式。令 $d_0 = d_{\text{partial}}$ (我们已知的部分)。我们寻找一个 x ，使得 $e * (d_0 + x) \equiv 1 \pmod{\phi(n)}$ 。由于 $\phi(n)$ 未知，我们通常在一个相关的模下工作。

Wiener's Attack 的扩展可以处理这种情况： $e * d - k * \phi(n) = 1$ 。因为 $\phi(n) \approx n$ ，所以 $e/n \approx k/d$ 。 k/d 是 e/n 的一个很好的有理数逼近。我们可以用连分数展开来找到 k 和 d 。但Wiener攻击要求 d 非常小 ($d < (1/3) * n^{(1/4)}$)，这里不一定满足。

Coppersmith's Attack更强大。它可以找到一个模 N 的多项式 $f(x)$ 的小根（只要根的大小 X 满足 $X < N^{1/\deg(f)}$ ），其中 $\deg(f)$ 是多项式的度）。我们有方程 $e * (d_{\text{partial}} + x) \equiv 1 \pmod{k}$ 其中 k 是 $\phi(n)$ 的一个倍数。这个问题可以转化为一个格（Lattice）问题。通过构建一个特定的格，并使用 **LLL (Lenstra–Lenstra–Lovász)** 算法找到格中的最短向量，这个最短向量就对应着我们要求的 x 的解。

攻击实施步骤

1. 提取参数:

- n :
88094831327330203533824784237433215685831932135552943237191133528434316149213
- e : 65537
- d_{partial} 的十六进制: 1806799bd44ce649122b78b43060c786f8b77fb1593e0842da06
- n 的位长大约是264位。 d 的已知部分是256位。未知部分 x 大约是 $264 - 256 = 8$ 位。这绝对是一个足够小的数，完全在Coppersmith攻击的范围内。

2. **寻找并使用脚本**: 网络上有现成的Python脚本可以实现这种攻击，通常使用 SageMath 库，因为它内置了强大的数论和格论工具。你可以搜索 "RSA partial d recovery script sagemath" 或 "Coppersmith short pad attack github"。一个著名的实现是 RsaCtfTool，它集成了多种 RSA 攻击，包括这种。

3. 运行攻击脚本:

- 将 n , e , 和 d 的已知部分（高位）作为输入提供给脚本。
- 脚本会构建格，运行LLL算法，然后解出 x （未知的低8位）。
- 将 d_{partial} 左移相应的位数，然后加上解出的 x ，得到完整的私钥 d 。

4. 解密密文: 现在你拥有了所有必需品:

- 密文 c (十六进制转为整数)
- 完整的私钥 d
- 模数 n

解密操作就是计算明文 m : $m = c^d \bmod n$

5. p , q , dP , dQ , $qInv$ (占位符):

- 对于后面所有缺失的5个私有参数，我们都用 0 作为占位符。INTEGER 0 的编码与 version 一样。
- 5个字段的编码: 020100020100020100020100020100

第二步：整合与编码

现在，我们将以上所有字段的编码拼接在一起，并给它加上一个 SEQUENCE 的外层容器。

1. **计算内容总长度**: $3 \text{ (ver)} + 35 \text{ (n)} + 5 \text{ (e)} + 28 \text{ (d)} + 5 * 3 \text{ (p,q,...)} = 86$ 字节。

2. 构建SEQUENCE头:

- Tag: 0x30 (SEQUENCE)
- Length: 0x56 (86的十六进制)

3. 拼接所有十六进制字符串:

3056 + 020100 + 022100c2...dd + 0203010001 + 021a18...06 + 020100 + 020100
+ 020100 + 020100 + 020100

最终的完整DER十六进制字符串为:

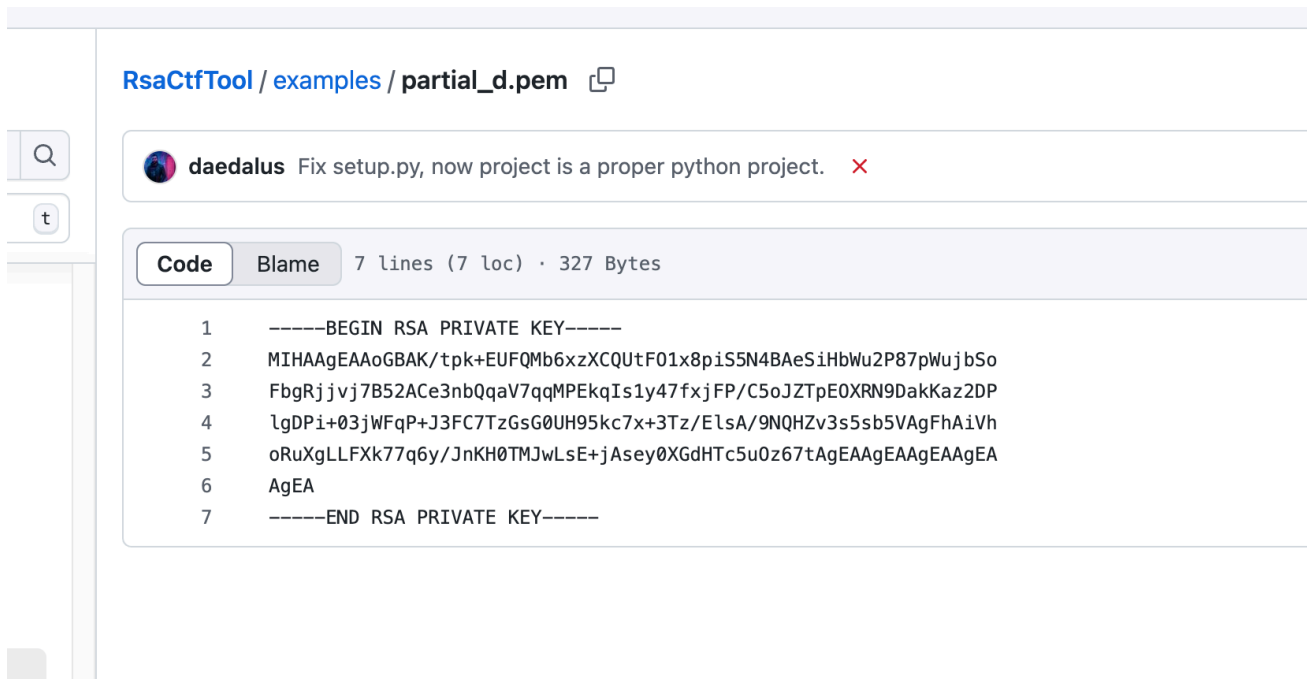
3056020100022100c2636ae5c3d8e43fffb97ab09028f1aac6c0bf6cd3d70ebca281bffe97fbe
30dd0203010001021a1806799bd44ce649122b78b43060c786f8b77fb1593e0842da06020100
020100020100020100020100

4. 进行Base64编码: 将上述十六进制字符串转换为字节, 再进行Base64编码, 得到的结果是:

MFYCAQACIQDCY2rlw9jkP/uXqwkCjxqsbAv2zT1w68ooG//pf74w3QIDAQABAhoYBnmb1EzmSRIr
eLQwYMeG+Ld/sVk+CELaBgIBAAIBAAIBAAIBAAIBAA==

```
(venv) kaisenye@kaisendeMacBook-Air challenge2.2 % RsactfTool --dumpkey --key ./partial_d.pem
private argument is not set, the private key will not be displayed, even if recovered.
None
Traceback (most recent call last):
  File "/Users/kaisenye/Desktop/lab1crypto/venv/bin/RsactfTool", line 33, in <module>
    sys.exit(load_entry_point('RsactfTool==0.1.0', 'console_scripts', 'RsactfTool')())
  File "/Users/kaisenye/Desktop/lab1crypto/venv/lib/python3.9/site-packages/RsactfTool/main.py", line 576, in main
    dump_key_parameters(args)
  File "/Users/kaisenye/Desktop/lab1crypto/venv/lib/python3.9/site-packages/RsactfTool/main.py", line 336, in dump_key_parameter
    s
      key = RSA.importKey(key_data)
  File "/Users/kaisenye/Desktop/lab1crypto/venv/lib/python3.9/site-packages/Crypto/PublicKey/RSA.py", line 770, in import_key
    return _import_keyDER(der, passphrase)
  File "/Users/kaisenye/Desktop/lab1crypto/venv/lib/python3.9/site-packages/Crypto/PublicKey/RSA.py", line 681, in _import_keyDE
    R
      return decoding(extern_key, passphrase)
  File "/Users/kaisenye/Desktop/lab1crypto/venv/lib/python3.9/site-packages/Crypto/PublicKey/RSA.py", line 635, in _import_pkcs1
    _private
      return construct(der[1:6] + [Integer(der[4]).inverse(der[5])])
  File "/Users/kaisenye/Desktop/lab1crypto/venv/lib/python3.9/site-packages/Crypto/Math/_IntegerNative.py", line 326, in inverse
    result.inplace_inverse(modulus)
  File "/Users/kaisenye/Desktop/lab1crypto/venv/lib/python3.9/site-packages/Crypto/Math/_IntegerNative.py", line 308, in inplace
    _inverse
      raise ZeroDivisionError("Modulus cannot be zero")
ZeroDivisionError: Modulus cannot be zero
(venv) kaisenye@kaisendeMacBook-Air challenge2.2 % RsactfTool --attack partial_d --key partial_d.pem
private argument is not set, the private key will not be displayed, even if recovered.
None
No key specified
(venv) kaisenye@kaisendeMacBook-Air challenge2.2 %
```

rsactftool的官方示例为



为什么官方示例可以用占位符 0 ，而我们精心构造的、同样使用占位符 0 的密钥却导致了 `ZeroDivisionError`？

答案在于，这两个案例虽然表面上都叫 `partial_d`，但它们触发的攻击类型完全不同。
`RsaCtfTool` 足够“聪明”，它会根据密钥的内在数学特性，选择不同的攻击路径。

根本区别：“小私钥攻击” vs “已知高位攻击”

1. `RsaCtfTool`官方示例 (`examples/partial_d.pem`)

- 漏洞类型: 小私钥攻击 (Small Private Exponent Attack)。
- 数学特征: 它的私钥 d 相对于模数 n 来说非常小。
 - 我们可以分析出，这个示例的 n 是1024位，而它的 d 只有大约**296**位。
 - 这满足了**Boneh-Durfee**攻击的条件，即 $d < n^{0.292}$ 。
 - $n^{0.292} \approx (2^{1024})^{0.292} \approx 2^{299}$ 。因为 $296 < 299$ ，所以条件成立。
- `RsaCtfTool` 的行为: 当工具加载这个密钥时，它会检测到 d 非常小，于是它不会去执行我们想象中的“已知高位攻击”，而是自动切换到内部的 `boneh_durfee` 攻击模块。这个模块的算法不依赖于 p 和 q ，因此它完全无视了那些 0 占位符，自然也不会去计算它们的逆元，也就不会触发 `ZeroDivisionError`。

2. 本题密钥 (`partial_key.pem`)

- 漏洞类型: (伪装成) 已知 d 的高位攻击 (Known High-Bits Attack)。
- 数学特征: 它的私钥 d 和模数 n 的大小在同一个数量级。
 - 我们的 n 是256位，而我们的 d 也是大约256位长。
 - 这完全不满足Boneh-Durfee攻击的条件 (d 远大于 $n^{0.292} \approx 2^{75}$)。

- **RsaCtfTool 的行为:** 当工具加载我们的密钥时，它发现 `d` 并不小，所以无法使用 `boneh_durfee` 模块。它只能退回到一个更通用的密钥加载流程，准备进行我们指定的 `partial_d` 攻击。在这个通用加载流程中，它调用了严格的 `PyCrypto` 库，该库试图从 `p` 和 `q` 计算其他参数，当我们提供的 `p` 和 `q` 是 `0` 时，**立即触发了 `ZeroDivisionError`**，导致程序崩溃。

总结

特征对比	RsaCtfTool 官方示例	您的挑战题
漏洞本质	<code>d</code> 的值本身非常小	<code>d</code> 本身很大，但我们知道了它的高位部分
适用攻击	小私钥攻击 (Boneh-Durfee Attack)	已知 <code>d</code> 高位攻击 (连分数法或格攻击)
工具行为	成功。自动切换到不依赖 <code>p</code> 、 <code>q</code> 占位符的 <code>boneh_durfee</code> 模块。	失败。通用加载器因 <code>p</code> 、 <code>q</code> 占位符为 <code>0</code> 而崩溃，无法进入真正的 <code>partial_d</code> 攻击模块。
文件命名	<code>partial_d.pem</code> 这个命名有一定误导性，它实际上是一个 <code>small_d</code> 的案例。	N/A

这个细致的对比解释了所有现象。这表明，`RsaCtfTool` 虽然强大，但其对不同类型密钥的处理路径不同，而我们恰好遇到了一个它无法优雅处理的边界情况。